unaffected when that function is `id`.[11] Even more interestingly, given `map(y)(id) == y`, we can perform the substitutions in the other direction to get back our original, more complex law. (Try it!) Logically, we have the freedom to do so because `map` can't possibly behave differently for different function types it receives. Thus, given `map(y)(id) == y`, it must be true that `map(unit(x))(f) == unit(f(x))`. Since we get this second law or theorem for free, simply because of the parametricity of `map`, it's sometimes called a *free theorem*.[12]

EXERCISE 7.7

Hard: Given `map(y)(id) == y`, it's a free theorem that `map(map(y)(g))(f) == map(y)(f compose g)`. (This is sometimes called *map fusion*, and it can be used as an optimization—rather than spawning a separate parallel computation to compute the second mapping, we can fold it into the first mapping.)[13] Can you prove it? You may want to read the paper "Theorems for Free!" (http://mng.bz/Z9f1) to better understand the "trick" of free theorems.

7.4.2 The law of forking

As interesting as all this is, this particular law doesn't do much to constrain our implementation. You've probably been assuming these properties without even realizing it (it would be strange to have any special cases in the implementations of `map`, `unit`, or `ExecutorService.submit`, or have `map` randomly throwing exceptions). Let's consider a stronger property—that `fork` should not affect the result of a parallel computation:

```
fork(x) == x
```

This seems like it should be obviously true of our implementation, and it is clearly a desirable property, consistent with our expectation of how `fork` should work. `fork(x)` should do the same thing as `x`, but asynchronously, in a logical thread separate from the main thread. If this law didn't always hold, we'd have to somehow know when it was safe to call without changing meaning, without any help from the type system.

Surprisingly, this simple property places strong constraints on our implementation of `fork`. After you've written down a law like this, take off your implementer hat, put on your debugger hat, and try to break your law. Think through any possible corner cases, try to come up with counterexamples, and even construct an informal proof that the law holds—at least enough to convince a skeptical fellow programmer.

---

[11] We say that `map` is required to be *structure-preserving* in that it doesn't alter the structure of the parallel computation, only the value "inside" the computation.

[12] The idea of free theorems was introduced by Philip Wadler in the classic paper "Theorems for Free!" (http://mng.bz/Z9f1).

[13] Our representation of `Par` doesn't give us the ability to implement this optimization, since it's an opaque function. If it were reified as a data type, we could pattern match and discover opportunities to apply this rule. You may want to try experimenting with this idea on your own.